

Gerador de Currículo:

salvando no banco de dados

A última aula do módulo leva o gerador para um banco **SQLite**: salvar vários currículos, abrir um salvo para editar, atualizar e excluir. Você vê como guardar o currículo inteiro como **JSON**, como preencher o formulário a partir do banco e como o padrão do Todo List (aula 12) se repete num projeto diferente.

SQLite + JSON

Criar · abrir · atualizar · excluir

Widgets com key

st.sidebar

st.toast

Dados pessoais

Formação Claude Code 2026 · Baseado em docs.python.org (sqlite3, json), docs.streamlit.io (Session State, Widget keys, st.toast) e cursor.com/docs (Agent, Plan Mode), consultados em outubro de 2026 (Streamlit 1.65) · Código de referência testado em projeto/ · Telas: print real

Da versão 1 *para a versão 2*

ESCOPO DA V2

- ✓ Botão **Salvar no banco**: cria na primeira vez, atualiza nas seguintes
- ✓ Barra lateral com os currículos salvos, do mais recente para o mais antigo
- ✓ Clicar num currículo abre os dados no formulário
- ✓ **Novo currículo** limpa o formulário
- ✓ Excluir com o ícone 🗑
- ✓ Prévia e PDF como na v1

O QUE SE REPETE DA AULA 12

Um `db.py` com uma função por operação, `sqlite3` da biblioteca padrão, consultas com `?`, `commit()` em cada escrita e o arquivo do banco no `.gitignore`.

O QUE É NOVO

Guardar um **objeto inteiro** (o currículo) numa coluna JSON, **atualizar** um registro existente e **preencher o formulário** a partir do banco.

Colunas *ou* JSON?

	UMA COLUNA POR CAMPO	UMA COLUNA JSON
Tabela	nome, email, telefone, cidade, resumo...	id, nome, dados, atualizado_em
Campo novo	Precisa alterar a tabela (migração)	Só aparece no JSON
Buscar por cidade no SQL	Simples	Possível, mas menos direto
Código	Mais SQL	<code>json.dumps</code> / <code>json.loads</code>

O código de referência usa **JSON**: o currículo é sempre lido e gravado inteiro, e campos novos (foto, idiomas) não exigem mexer na tabela. O **nome** fica também numa coluna própria, para a lista lateral.

Antes de codar, compare guardar o currículo em colunas separadas ou numa coluna JSON, para este app. Recomende uma opção e explique a troca.

Decisão de modelagem é ótima para o modo **Ask** ou **Plan**: discuta antes, codifique depois.

O pedido *da versão 2*

Salve os currículos num SQLite (currículos.db):

- db.py com conectar (cria a tabela), listar, carregar, salvar (cria ou atualiza pelo id) e excluir; o currículo vai como JSON e o nome também numa coluna, com data de atualização;
- barra lateral: "Novo currículo" e a lista dos salvos (abrir com clique, excluir com 🗑);
- os campos do formulário passam a usar key, para serem preenchidos ao abrir um currículo;
- botão "Salvar no banco" ao lado do PDF, com um aviso de "Currículo salvo!".

Mantenha prévia, validação e PDF.

TRECHO	POR QUÊ
"cria ou atualiza pelo id"	Salvar duas vezes não duplica o currículo
"nome também numa coluna"	Listar sem abrir cada JSON
"campos passam a usar key"	É o que permite preencher o formulário pelo código
"Mantenha..."	Protege a v1

db.py *dos currículos*

```
def listar(conexao):
    return conexao.execute(
        "SELECT id, nome, atualizado_em FROM curriculos ORDER BY atualizado_em DESC, id DESC"
    ).fetchall()

def carregar(conexao, id_curriculo):
    linha = conexao.execute("SELECT dados FROM curriculos WHERE id = ?", (id_curriculo,)).fetchone()
    return json.loads(linha["dados"]) if linha else None

def salvar(conexao, dados, id_curriculo=None):
    """Cria um currículo novo ou atualiza o existente. Devolve o id."""
    texto = json.dumps(dados, ensure_ascii=False)
    if id_curriculo is None:
        cursor = conexao.execute("INSERT INTO curriculos (nome, dados) VALUES (?, ?)", (dados["nome"], texto))
        id_curriculo = cursor.lastrowid
    else:
        conexao.execute(
            "UPDATE curriculos SET nome = ?, dados = ?, atualizado_em = datetime('now', 'localtime') WHERE id = ?",
            (dados["nome"], texto, id_curriculo),
        )
    conexao.commit()
    return id_curriculo

def excluir(conexao, id_curriculo):
    conexao.execute("DELETE FROM curriculos WHERE id = ?", (id_curriculo,))
    conexao.commit()
```

- ✓ `conectar()` cria a tabela `curriculos` (`id, nome, dados, atualizado_em`) se ainda não existir, como na aula 12
- ✓ `ensure_ascii=False` guarda "São Paulo" legível no JSON
- ✓ `salvar()` : sem id faz `INSERT` e devolve o id novo (`lastrowid`); com id faz `UPDATE`
- ✓ O `UPDATE` renova `atualizado_em`, para a lista ficar em ordem
- ✓ Tudo com `?` : nomes com apóstrofo (D'Ávila) não quebram nada

Preenchendo o formulário *peelo código*

```
CAMPOS = ["nome", "email", "telefone", "cidade", "resumo", "experiencias", "formacao", "habilidades"]
conexao = db.conectar()

if "id_atual" not in st.session_state:
    st.session_state.id_atual = None # None = currículo novo, ainda não salvo

def preencher(dados, id_curriculo):
    for campo in CAMPOS:
        st.session_state[campo] = dados.get(campo, "")
    st.session_state.id_atual = id_curriculo

# Barra lateral: currículos salvos
with st.sidebar:
    st.header("Currículos salvos")
    if st.button("✚ Novo currículo", width="stretch"):
        preencher({}, None)
        st.rerun()
    salvos = db.listar(conexao)
    if not salvos:
        st.caption("Nenhum currículo salvo ainda.")
    for linha in salvos:
        atual = linha["id"] == st.session_state.id_atual
        col_nome, col_excluir = st.columns([4, 1], vertical_alignment="center")
        rotulo = f"▶ ' if atual else ''}{linha['nome']}"
        if col_nome.button(rotulo, key=f"abrir_{linha['id']}", help=f"Salvo em {linha['atualizado_em']}",
                           width="stretch"):
            preencher(db.carregar(conexao, linha["id"]), linha["id"])
            st.rerun()
        if col_excluir.button("🗑", key=f"excluir_{linha['id']}", help="Excluir"):
            db.excluir(conexao, linha["id"])
            if atual:
                preencher({}, None)
            st.rerun()
```

O TRUQUE DAS KEYS

Cada campo do formulário tem `key="nome"`, `key="email"` ... O valor do campo mora em `st.session_state["nome"]`. Para "abrir" um currículo, `preencher()` escreve nessas chaves e chama `st.rerun()`: os campos aparecem preenchidos.

Só dá para escrever na chave de um campo **antes** de ele ser desenhado na execução. Aqui funciona porque a barra lateral roda antes do formulário. Se o agente inverter a ordem, o Streamlit acusa erro.

`id_atual` lembra qual currículo está aberto: `None` = novo; um número = o salvo que será atualizado.

Salvar *sem duplicar*

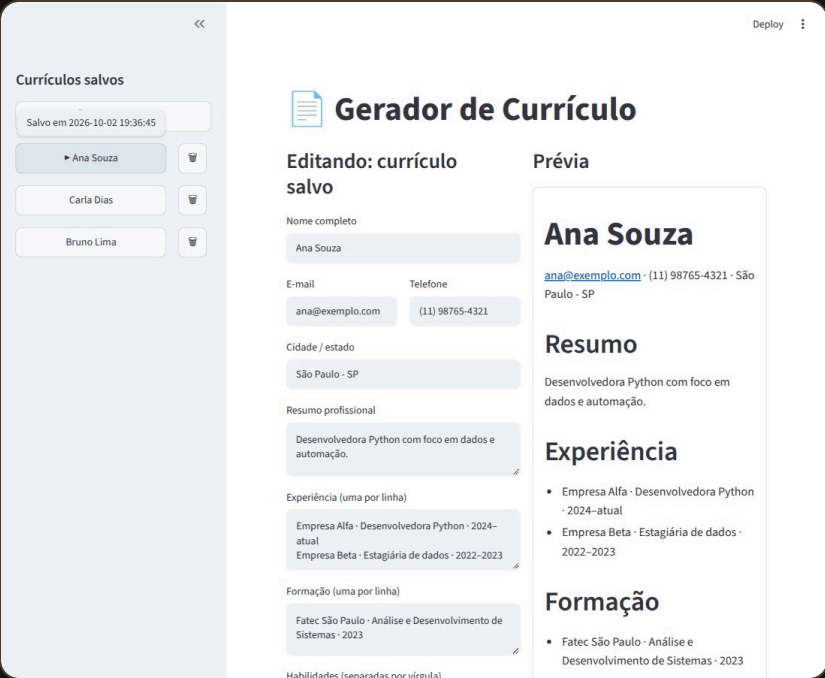
```
dados = {campo: st.session_state.get(campo, "").strip() for campo in CAMPOS}

with col_previa:
    st.subheader("Prévia")
    problemas = []
    if not dados["nome"]:
        problemas.append("Preencha o nome.")
    if dados["email"] and not re.fullmatch(r"^[^@\\s]+@[^@\\s]+\\.^[^@\\s]+$", dados["email"]):
        problemas.append("O e-mail parece inválido.")

    if problemas:
        for p in problemas:
            st.warning(p)
    else:
        with st.container(border=True):
            st.markdown(para_markdown(dados))
        col_salvar, col_pdf = st.columns(2)
        if col_salvar.button("💾 Salvar no banco", type="primary", width="stretch"):
            st.session_state.id_atual = db.salvar(conexao, dados, st.session_state.id_atual)
            st.toast("Currículo salvo!")
            st.rerun()
        arquivo = re.sub(r"\\W+", "_", dados["nome"].lower()).strip("_") or "curriculo"
        col_pdf.download_button("Baixar PDF", para_pdf(dados), f"curriculo_{arquivo}.pdf",
                                "application/pdf", width="stretch")
```

- ✓ Os dados vêm das chaves do `session_state`
- ✓ Salvar passa o `id_atual`: o mesmo currículo é atualizado, não duplicado
- ✓ Depois do primeiro salvamento, `id_atual` passa a ser o id novo
- ✓ `st.toast` mostra "Currículo salvo!" e continua visível depois do `st.rerun()`
- ✓ Validação e PDF são os da v1

A versão 2 *funcionando*



Print real do projeto/ com três currículos fictícios num curriculos.db de exemplo; o da Ana está aberto

Roteiro *de teste*

TESTE	ESPERADO
Salvar um currículo novo	Aparece na barra lateral; aviso "Currículo salvo!"
Mudar a cidade e salvar de novo	Continua um registro, com a cidade nova
Novo currículo	Formulário limpo; o salvo continua na lista
Fechar e abrir o app; clicar num salvo	Os dados voltam ao formulário
Excluir o que está aberto	Some da lista; formulário limpo
Nome com apóstrofo	Salva e abre normalmente

O código de referência passou nos cinco primeiros com o **AppTest** : salvar duas vezes manteve 1 registro; "Novo" limpou o formulário; numa sessão nova, abrir a Ana trouxe "Campinas - SP"; excluir deixou só o Bruno.

@Browser Abra o gerador, salve um currículo fictício, altere a cidade e salve de novo. Quantos itens aparecem na barra lateral? Recarregue a página, abra o currículo e confira a cidade.

Currículo é *dado pessoal*

- ✓ `curriculos.db` no `.gitignore`: nunca vai para o repositório
- ✓ Teste com **dados fictícios**, como os do print
- ✓ Excluir pede só um clique: para uso real, peça uma **confirmação**
- ✓ Backup = copiar o arquivo; guarde num lugar seguro

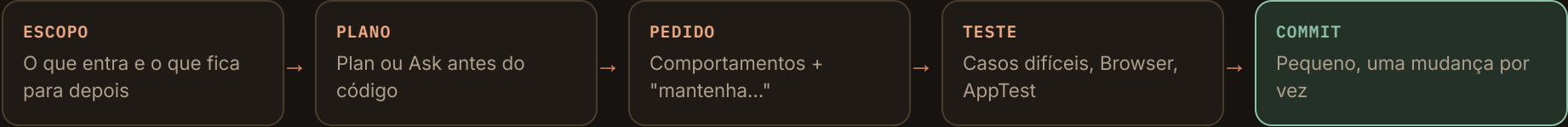
```
# .gitignore
.venv/
__pycache__/
curriculos.db
```

Se este app fosse publicado na internet, qualquer pessoa veria e apagaria os currículos de todo mundo. Antes de publicar, ele precisaria de **login** e de separar os dados por usuário (e, no Brasil, seguir a **LGPD**).

Ao pedir ao agente para consultar o banco (no Ask ou no terminal), lembre que os dados vão para o contexto do modelo. Com dados reais, prefira olhar você mesmo.

Quatro projetos, *um método*

PROJETO	AULAS	O QUE VOCÊ PRATICOU
ToDo List	10 a 12	Escopo, rule, Plan, chaves estáveis, SQLite parametrizado
Pedra, Papel e Tesoura	13 a 15	Regra separada da tela, 9 combinações, mock, imagens, ordem de execução
Analizador de Texto	16 a 18	Regex, Counter, dependência nova, nuvem, stopwords em arquivo
Gerador de Currículo	19 e 20	Formulário, PDF, erros de biblioteca, JSON no banco, dados pessoais



Teste o que *ficou*

1. Por que salvar() recebe o id do currículo aberto?

A) Para apagar o antigo

B) Para atualizar em vez de criar outro registro

C) O SQLite exige

D) Para gerar o PDF

B. Sem id = INSERT; com id = UPDATE.

2. O que permite "abrir" um currículo no formulário?

A) Recarregar a página

B) Campos com key + escrever em st.session_state antes de desenhá-los

C) Um iframe

D) O PDF

B. E um st.rerun() em seguida.

3. Uma vantagem de guardar o currículo como JSON?

A) Buscas SQL mais simples

B) Campos novos sem alterar a tabela

C) Ocupa zero espaço

D) Criptografa os dados

B. Em troca, buscar por campo é menos direto.

4. O que falta para publicar este app na internet?

A) Nada

B) Login e dados separados por usuário

C) Mais cores

D) Outro banco

B. Currículo é dado pessoal.

Do agente *ao produto*

O gerador agora salva, abre, atualiza e exclui currículos num **SQLite**, com o currículo inteiro em **JSON** e o formulário preenchido pelas **keys**. Neste módulo, você conheceu o Agent, a voz, o Ask, as imagens, o MCP (do protocolo ao GitHub e ao navegador) e construiu quatro projetos com o mesmo método: escopo, plano, pedido, teste e commit.

SQLite + JSON

Criar · abrir · atualizar · excluir

Widgets com key

Dados pessoais

Escopo → commit

Fim do módulo 36 · Funcionalidades de IA e MCP no Cursor

Fontes: docs.python.org (sqlite3, json); docs.streamlit.io (Session State, Widget keys, st.sidebar, st.toast, AppTest); cursor.com/docs (Agent Overview, Plan Mode, Browser), consultados em outubro de 2026. Código de referência em [projeto/](#), testado com Streamlit 1.65 e fpdf2 2.8.9.